

Project 2. Logic - Futoshiki Puzzles

Report

Team members: StudentID1, StudentID2. Contribution: StudentID1 50%, StudentID2 50%.

Video URLs: add YouTube or Google Drive links after recording the required demonstrations.

Self-Evaluation

Criterion	Status
FOL formalization	Included below with complete rules and three CNF derivations.
Automatic KB/CNF generation	Implemented in kb_generator.py and cnf_generator.py.
Forward chaining	Implemented with Horn facts/rules plus deterministic propagation.
Backward chaining	Implemented with unification-based Horn query helper and SLD-style DFS search.
A* search	Implemented with admissible remaining-cells heuristic and metrics.
Brute force and backtracking	Implemented and compared on all ten inputs.
Experiments	Runtime, memory, nodes, assignments, inferences, and frontier are reported.
GUI	Optional Tkinter GUI implemented in gui.py.

Formal FOL Axioms

Domain: row, column, and value variables range over $\{1, \dots, N\}$.

A1: For all i, j , exists v such that $\text{Val}(i, j, v)$.

A2: For all i, j, v_1, v_2 , $\text{Val}(i, j, v_1)$ and $\text{Val}(i, j, v_2)$ implies $v_1 = v_2$.

A3: For all i, j_1, j_2, v , $\text{Val}(i, j_1, v)$ and $\text{Val}(i, j_2, v)$ and $j_1 \neq j_2$ implies contradiction.

A4: For all j, i_1, i_2, v , $\text{Val}(i_1, j, v)$ and $\text{Val}(i_2, j, v)$ and $i_1 \neq i_2$ implies contradiction.

A5: For all i, j, v , $\text{Given}(i, j, v)$ implies $\text{Val}(i, j, v)$.

A6: For all i, j, v_1, v_2 , $\text{LessH}(i, j)$ and $\text{Val}(i, j, v_1)$ and $\text{Val}(i, j+1, v_2)$ implies $\text{Less}(v_1, v_2)$.

A7: For all i, j, v_1, v_2 , $\text{GreaterH}(i, j)$ and $\text{Val}(i, j, v_1)$ and $\text{Val}(i, j+1, v_2)$ implies $\text{Less}(v_2, v_1)$.

A8: For all i, j, v_1, v_2 , $\text{LessV}(i, j)$ and $\text{Val}(i, j, v_1)$ and $\text{Val}(i+1, j, v_2)$ implies $\text{Less}(v_1, v_2)$.

A9: For all i, j, v_1, v_2 , $\text{GreaterV}(i, j)$ and $\text{Val}(i, j, v_1)$ and $\text{Val}(i+1, j, v_2)$ implies $\text{Less}(v_2, v_1)$.

A10: For every value v in the finite domain, each row and each column contains v at least once.

CNF Derivations

Given enforcement: $\text{Given}(i, j, v) \rightarrow \text{Val}(i, j, v)$. Eliminate implication: not $\text{Given}(i, j, v)$ or $\text{Val}(i, j, v)$. Clause: $\{\sim \text{Given}(i, j, v), \text{Val}(i, j, v)\}$.

At most one value: $\text{Val}(i, j, v_1)$ and $\text{Val}(i, j, v_2) \rightarrow \text{false}$ for $v_1 \neq v_2$. Eliminate implication: not $(\text{Val}(i, j, v_1) \text{ and } \text{Val}(i, j, v_2))$. Distribute: $\{\sim \text{Val}(i, j, v_1), \sim \text{Val}(i, j, v_2)\}$.

Horizontal less: $\text{LessH}(i, j)$ and $\text{Val}(i, j, v_1)$ and $\text{Val}(i, j+1, v_2) \rightarrow \text{Less}(v_1, v_2)$. For violating pairs $v_1 \geq v_2$, replace consequent by false, giving clause $\{\sim \text{LessH}(i, j), \sim \text{Val}(i, j, v_1), \sim \text{Val}(i, j+1, v_2)\}$.

Row uniqueness: $\text{Val}(i, j_1, v)$ and $\text{Val}(i, j_2, v)$ and $j_1 \neq j_2 \rightarrow \text{false}$. Since $j_1 \neq j_2$ is fixed after grounding, each pair $j_1 < j_2$ creates $\{\sim \text{Val}(i, j_1, v), \sim \text{Val}(i, j_2, v)\}$.

Algorithm Summaries

Forward chaining: initialize Given and constraint facts, apply Horn rules until fixed point, update singleton domains and hidden singles, and detect empty domains as contradictions. If propagation stalls, branch on the smallest domain.

Backward chaining: query $\text{Val}(i, j, v)$ goals with a unification-based Horn helper, choose the unassigned cell with the smallest provable domain, and use depth-first SLD-style search.

A*: a state is a partial assignment. $g(s)$ is the number of assignments added after the initial state. $h(s)$ is the number of unassigned cells. Each action fills one cell, so h never overestimates the minimum remaining assignments and is admissible.

Backtracking uses MRV for variable selection. Brute force fills cells in row-major order with only local consistency pruning, so it is a weaker baseline.

Input Coverage

Input	N	Clues	Inequality constraints
input-01.txt	4	3	10
input-02.txt	4	8	10
input-03.txt	5	15	17
input-04.txt	5	15	17
input-05.txt	6	24	22
input-06.txt	6	22	22
input-07.txt	7	35	28
input-08.txt	7	25	28
input-09.txt	9	71	36
input-10.txt	9	54	36

Experiment Results

Input	N	Method	Solved	Time(s)	Peak KB	Nodes	Assign	Infer	Frontier
input-01.txt	4	bruteforce	True	0.000283	2.7	20	55	0	0
input-01.txt	4	backtrack	True	0.000810	3.9	14	13	0	0
input-01.txt	4	forward	True	0.001758	27.2	1	0	76	0
input-01.txt	4	backward	True	0.002336	16.0	14	13	13	0
input-01.txt	4	astar	True	0.001371	6.3	14	13	0	1
input-02.txt	4	bruteforce	True	0.000156	2.1	9	24	0	0
input-02.txt	4	backtrack	True	0.000308	1.9	9	8	0	0
input-02.txt	4	forward	True	0.001429	21.9	1	0	81	0
input-02.txt	4	backward	True	0.001251	11.3	9	8	8	0
input-02.txt	4	astar	True	0.000638	4.4	9	8	0	1
input-03.txt	5	bruteforce	True	0.000198	3.1	11	30	0	0
input-03.txt	5	backtrack	True	0.000467	2.9	11	10	0	0
input-03.txt	5	forward	True	0.004101	43.2	1	0	155	0
input-03.txt	5	backward	True	0.003425	15.1	11	10	10	0
input-03.txt	5	astar	True	0.001915	8.1	11	10	0	1
input-04.txt	5	bruteforce	True	0.000365	3.1	11	32	0	0
input-04.txt	5	backtrack	True	0.000699	2.9	11	10	0	0
input-04.txt	5	forward	True	0.002168	36.0	1	0	155	0
input-04.txt	5	backward	True	0.001952	14.9	11	10	10	0
input-04.txt	5	astar	True	0.001050	8.1	11	10	0	1
input-05.txt	6	bruteforce	True	0.000244	3.1	13	54	0	0
input-05.txt	6	backtrack	True	0.000694	3.1	13	12	0	0
input-05.txt	6	forward	True	0.003159	47.3	1	0	258	0
input-05.txt	6	backward	True	0.002787	26.4	13	12	12	0
input-05.txt	6	astar	True	0.001459	10.1	13	12	0	1
input-06.txt	6	bruteforce	True	0.000270	3.2	15	34	0	0

Input	N	Method	Solved	Time(s)	Peak KB	Nodes	Assign	Infer	Frontier
input-06.txt	6	backtrack	True	0.000872	3.1	15	14	0	0
input-06.txt	6	forward	True	0.003081	36.9	1	0	256	0
input-06.txt	6	backward	True	0.003573	25.7	15	14	14	0
input-06.txt	6	astar	True	0.001766	10.7	15	14	0	1
input-07.txt	7	bruteforce	True	0.000338	3.1	15	77	0	0
input-07.txt	7	backtrack	True	0.000943	3.2	15	14	0	0
input-07.txt	7	forward	True	0.003894	78.1	1	0	401	0
input-07.txt	7	backward	True	0.004222	29.9	15	14	14	0
input-07.txt	7	astar	True	0.001981	12.7	15	14	0	1
input-08.txt	7	bruteforce	True	0.000367	3.5	25	84	0	0
input-08.txt	7	backtrack	True	0.002603	4.3	25	24	0	0
input-08.txt	7	forward	True	0.004275	76.0	1	0	391	0
input-08.txt	7	backward	True	0.008390	40.6	25	24	24	0
input-08.txt	7	astar	True	0.004643	18.6	25	24	0	1
input-09.txt	9	bruteforce	True	0.000361	4.7	11	82	0	0
input-09.txt	9	backtrack	True	0.000765	5.2	11	10	0	0
input-09.txt	9	forward	True	0.007848	126.5	1	0	830	0
input-09.txt	9	backward	True	0.004531	38.1	11	10	10	0
input-09.txt	9	astar	True	0.002180	15.7	11	10	0	1
input-10.txt	9	bruteforce	True	0.000500	5.3	29	144	0	0
input-10.txt	9	backtrack	True	0.004429	7.5	28	27	0	0
input-10.txt	9	forward	True	0.007756	92.9	1	0	813	0
input-10.txt	9	backward	True	0.014595	70.9	28	27	27	0
input-10.txt	9	astar	True	0.007069	28.7	28	27	0	1

Method	Average runtime (s)	Solved cases
astar	0.002407	10
backtrack	0.001259	10
backward	0.004706	10
bruteforce	0.000308	10
forward	0.003947	10

Comparative Analysis

The generated cases contain many clues, so all algorithms solve them quickly. Forward chaining often expands one search node because propagation derives enough singleton domains. A* and backtracking expand similar numbers of states because both use small-domain choices; A* additionally tracks a priority frontier. Brute force is slower in less constrained cases because it uses row-major order instead of MRV.

References

Course assignment handout: Project 2. Logic - Futoshiki Puzzles.